



EMERGING WORLD MODELS

Emerging World Models —

World models are one of the most active directions in AI research. EWM is the part we build: the memory.

Emerging World Models (EWM) attaches to the model you already run as a side-car — never as a replacement. The host keeps its weights, tokenizer, prompts and APIs, and gains a structural, content-addressed memory that is **measured, verifiable and portable across models**. Nothing inside the model is retrained, fine-tuned or replaced. It runs on-premise, inside your own boundary.

[How It Works](#)

[See the Evidence](#)

What a side-car is, in plain words

This word runs through the whole site, and it is not one everybody meets in their own line of work. A side-car is a design pattern — the name people who build software give



Start with the motorbike

A side-car is the extra seat bolted to the side of a motorbike. The bike is not taken apart to fit it: the engine, the wheels and the handlebars stay exactly as they were. The side-car simply travels alongside and adds something the bike never had — room for a passenger.

The same idea sits on things you already own

A phone case with a battery in it gives you days of charge without opening the phone. A rack on the back of a car carries bikes without turning it into a different car. An external drive gives a laptop more space without taking the laptop apart. In every case the main thing stays as it is, and something is attached beside it to add what was missing.

Why people build things this way

- ▶ Nothing that already works gets opened up, so there is no risk of breaking it.
- ▶ The add-on can be fitted, updated or removed on its own. The main thing never changes.
- ▶ Each part can be looked after and repaired separately.
- ▶ It is far quicker and cheaper than rebuilding the main thing to include the new ability.

What it costs you

- ▶ There is now one more thing to fit, watch over, update and keep secure.
- ▶ Passing information back and forth between the two takes a little extra time and effort.
- ▶ An add-on can only help with what the main thing lets it see. Whatever the main thing keeps to itself, the add-on cannot use.
- ▶ If something goes wrong, there are two parts to check instead of one.

Software is full of this move without ever using the name: a small helper attached to a large program to handle logging, security checks or monitoring, so the large program never has to be modified for them. It is the same idea as the battery case.



remember. It listens to the signals the model produces while it works — its internal encodings, in technical terms — keeps a record of them outside the model, and can hand similar signals back when the model needs them. The model itself is not opened up, retrained or replaced; it keeps its weights, its tokenizer and its usual behaviour. The same honest limit applies here as anywhere else: we can only work with the signals the model makes available to us.

Context and memory are two different jobs

The two words are used interchangeably in everyday speech, which is where most confusion about AI memory begins. They are related, their purposes overlap, and they are not the same thing.

Context — the working set

Everything the model can see at the moment it answers: instructions, the conversation so far, retrieved passages, tool output. A hosted model call is effectively independent of the one before it, so whatever the model conditions on has to travel in the request. Context is re-sent every turn, bounded by the window, and gone once the call returns.

Memory — the durable state

What a system keeps after the call is over and can find again on purpose: decisions, results, history, structure. It is not re-sent automatically and it does not expire with the call. Memory lives outside the model — in a store, an index, a database — and retrieval is the pipe that carries things from there into the context.



	single call.	and can be addressed later.
Lifetime	The length of the call.	Until something explicitly removes it.
What limits it	The context window.	Whatever your storage can hold.
Cost behaviour	Re-sent, and re-priced, on every turn.	Read when it is needed.
Structure	Text and vectors, in the order you assembled them.	Whatever the memory design imposes — often nothing.
How you check it	You can inspect what you sent.	Only if the store itself is checkable.

Why the two get conflated

Because in practice everything a model appears to remember ends up inside the context anyway. Context is the channel the model reads; memory is the place information lives between calls. Retrieval sits between them: memory → retrieval → context. That is why a system can have a very large window and still have no memory, and why adding retrieval does not by itself produce a memory you can audit.

Where systems usually intervene

- ▶ **A larger window.** More context per call; the state still has to be re-sent, and cost grows with it.
- ▶ **Summarisation.** Compresses what cannot be re-sent; structure — counts, overlaps, what is new — is what gets dropped in the process.
- ▶ **Retrieval.** Gives the state an address so it can be found again; on its own it ranks by similarity and guarantees no coverage.
- ▶ **Fine-tuning.** Moves information into the weights, where it is no longer separately addressable — and updating it means training again.



that window still matters. What the side-car adds is a home for the durable state outside the model, with a structure that can be checked: what is retained, what departed and what is new.

Where EWM sits in the world-model movement

World models are a bet much of the field is making — that a system should carry an internal, predictive model of its world, not only a larger next-token predictor. Latent predictive architectures such as JEPA, learned simulators and generative video models are all part of that programme, and it is being advanced by far more groups than ours.

We do not claim the thesis. We contribute one part of it: the memory.

A shared programme, not a contested one

We treat the wider world-model work as the ground EWM grows in. Our papers, our vocabulary and our guarantees are meant to sit alongside it — a layer other groups can use, including teams building predictive encoders and latent state models.

We add to your model, we do not replace it

The side-car reads the host's own internal encodings, keeps state outside the model and hands encodings back. Weights, tokenizer, prompts and APIs are untouched — adopting the memory never means retiring the model.

Model-agnostic on purpose

Because EWM works at the measurement level, one lattice spans model families and modalities: a JEPA-style predictive encoder, a vision stack,

What we do claim

That a memory layer beside a model can be measured, verified and carried across models — with no change to the model itself. That is a deliberately narrow claim, and the evidence for it is published in full.

The missing piece is memory, not intelligence

Today's models reason impressively, and the mechanisms we use to give them context work: a window re-sent each turn, summaries, retrieved chunks. What they are not is a memory you can measure. State between calls stays bounded, lossy and unstructured — and that is the gap a side-car closes without touching the model.

1

Bounded working context

Reasoning quality is capped by what fits in one window. Everything relevant must be re-sent every turn, so cost and latency scale with it. When the window fills, something is dropped — heuristically, not structurally — and the model cannot tell you what it lost.

2

Lossy, unstructured history

Long-term memory arrives as summaries, notes or retrieved chunks. Summarization discards structure: counts, overlaps and novelty vanish. Retrieval is similarity, not position in a lattice, so there is no guarantee of coverage.

3



what is new, what is retained and what departed. There is no algebra over model state — only text diffs and embeddings. Nothing is content-addressed, so nothing is independently verifiable.

How the side-car relates to all three

It does not remove the window, and it does not replace retrieval or summaries. It adds a structural state beside them: the model keeps reasoning exactly as it does today, and gains a record of what it retained, what departed and what is new.

See how the side-car works

Measured on real models, with the caveats published

Every figure below comes from a run in our own repositories — real model encodings, real documents, on our own machines. Where a number has a limit, the limit is printed beside it.

1.0

loop accuracy — hash-level round-trip on a real VLA

100%

word retention — ordered OCR reconstruction

32,768

measurement sites per state, content-addressed



model changes — no
retraining, no fine-tuning,
no new weights

All six measured scenarios

Published record

Clinical and diagnostic outputs are research demonstrations for partner evaluation. They are not a medical device and do not replace clinician review.

AI Research & Innovation — Products in Progress

Five products in progress, all built on the same measurement lattice as the side-car — language models, world models, vision, document intelligence and vision-language-action autonomy.



IMPLEMENTED

NanoLM

Language Models

Megabyte-scale domain LLM built on De Bruijn Markov chains — deterministic, CPU-only, with no GPU and no pretrained weights.



PROTOTYPE

EWM Japa

World Models

JEPA-style world-model memory service: frames are encoded into a deterministic patch codebook, stored as content-addressed HLLSet keys in the EWM lattice and recalled by Bell State Similarity — no

**IN PROGRESS**

Vision Studio

Vision & Perception

Studio runtime for the vision side-car — 16×16 patch grids through conv(n,2) encoders, frame-by-frame positional restoration, with the lattice's DRN decomposition and temporal pyramid as the inspection surface.

**IMPLEMENTED**

DeepSeek-OCR

Document Intelligence

Document extraction on the DeepSeek-OCR model with a CPU-only Tesseract fallback; drives the semantic compressor across a real 128K-vocabulary BPE boundary at 100% word retention and zero order loss.

**PROTOTYPE**

EWM VLA

Autonomous Systems & VLA

Vision-Language-Action side-car: visual embeddings, language tokens and action trajectories are mapped into HLLSet frames for content-addressable scene memory and lattice reasoning over perception-action chains — hash-level round-trip accuracy 1.0 on a real VLA host.

Partner with Our Research Team

What the side-car gives the model you already run



A Structure, Not a Summary

D / R / N per commit — departed, retained, novel — with a conservation balance. A precise answer to “what changed?”



The Model Is Untouched

Zero model changes — no retraining, no fine-tuning, no new weights. The host keeps its weights, tokenizer and APIs.



Auditable by Construction

Content-addressed, immutable commits give an append-only record — exactly what an auditor asks for.



Portable Across Models



On-Premise & Air-Gapped

Deployed inside your own boundary, including air-gapped environments with zero outbound calls. The state store sits in storage you control.



Proven on Real Models

Six measured scenarios across a vision-language-action stack, OCR, vision grids and a document platform.

Bring your own model — we bring the memory

Run the side-car loop against your own model as a design partner, pilot it on a real workload, or engage our team on a custom build. Your weights, tokenizer and APIs stay exactly as they are.



Technology

How It Works

Evidence & Benchmarks

Docs & Research

Trust & Security

Scenarios

Applications

Solutions

Products

NanoLM

Visual Memory

AI-PMS

SGS.ai

Company

About Us

Design Partner Program

Contact Us

Contact Us



Customer Support



+91 9845555760



© 2026 Codentic AI. All rights reserved.

[Privacy Policy](#) · [Terms of Use](#) · [Trust & Security](#)

Emerging World Models — a memory side-car beside any AI model, never a replacement for it.